Tutorial 8 — Mealy and Moore Machines, Setup and Hold Times, Registers

---

Q1 — Negating a Number One Bit at a Time

Question (discuss in class)

A sensor sends 6-bit two's complement numbers N over a single wire, one bit per clock cycle, least significant bit (LSB) first. A receiver has to send −N out on another wire, also one bit per clock and LSB first, without ever storing the whole number.

(a)

Fill in the table. Then compare N and −N bit by bit, starting from the LSB (the rightmost bit). Which bits are copied unchanged and which are inverted? State the rule in one sentence. How many bits of history does a circuit need to remember to apply your rule while the bits stream past?

N	N (6-bit)	−N (6-bit two's complement)
12		
5		
22		
1		
24		

(b)

Draw a two-state Mealy state diagram for your rule, with input X (incoming bit) and output Z (outgoing bit). Encode the states with one D flip-flop Q (Q = 0 in the start state) and derive D and Z.

(c)

Now convert your Mealy machine into a Moore machine by following these steps:

- (i) For each Mealy state, list every arrow that enters it and the output value written on that arrow. Treat the start state as entered with output 0.
- (ii) If all arrows entering a state carry the same output, it becomes one Moore state with that output. If they carry both 0 and 1, split it into two Moore states, one for each output value. Every copy keeps the same outgoing transitions as the original state. An arrow that used to enter the original state now goes to the copy whose output matches the arrow's output.
- (iii) Draw the Moore state diagram. How many states and flip-flops does it need?

(d)

Trace both machines for N = 22, starting from their start states. X carries the bits of 22, LSB first, in cycles 1 to 6. In cycle 7, X = 0, and you can ignore it. In which cycles does each machine output the six bits of −22? How many clock cycles does each one need to deliver the complete result?

Cycle	X	Mealy state	Mealy Z	Moore state	Moore Z
1	0				
2	1				
3	1				
4	0				
5	1				
6	0				
7	— (0)				

Q2 — How Fast Can the Wiper Controller Run?

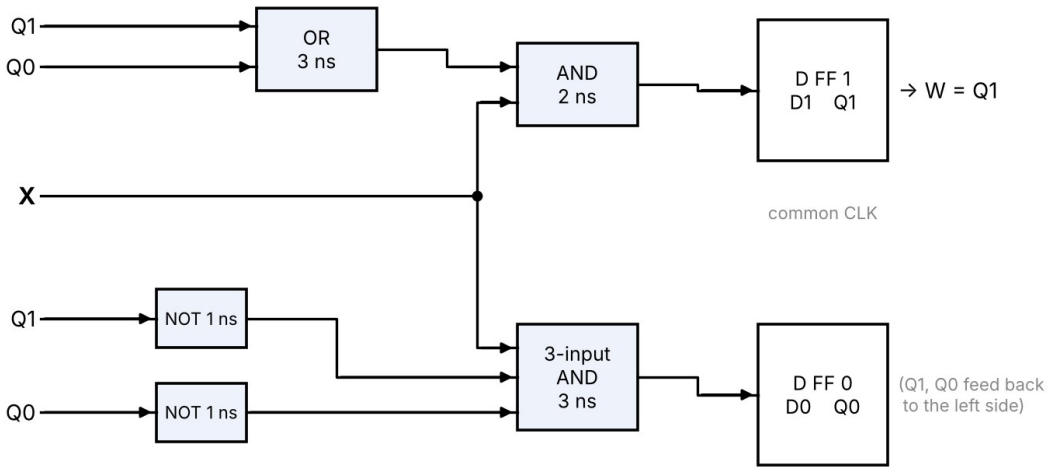
Question (discuss in class)

A car's rain sensor gives X = 1 when it detects water on the windscreen. A controller reads X at every rising clock edge and switches the wiper motor on (W = 1) only after rain has been seen at two consecutive edges, so a single stray drop does not start the wipers.

The controller stores its state in two positive-edge-triggered D flip-flops, Q1 Q0, which share one clock. State 00 means "no rain at the last edge", 01 means "rain at the last edge only", and 10 means "rain at the last two or more edges". State 11 is never used. The controller equations are:

```
D1 = X·(Q1 + Q0)        D0 = X·Q1'·Q0'        W = Q1
```

They are built from the gates below. Treat every delay as exact.



- Flip-flop: after a rising clock edge, Q takes 4 ns to change (clock-to-Q delay). D must be stable for at least 3 ns before the edge (setup time) and must stay stable for at least 1 ns after it (hold time). If D changes inside this window, the stored value is unpredictable.
- Gates: NOT 1 ns, 2-input AND 2 ns, 2-input OR 3 ns, 3-input AND 3 ns.

(a)

A rising edge occurs at t = 0, and X is steady. Fill in the time at which each signal settles to its new value. What is the shortest clock period for which both D1 and D0 meet the setup time at the next edge? What is the highest clock frequency?

Signal	Settles at (ns after the edge)
Q1, Q0	
Q1', Q0' (inverter outputs)	
Q1+Q0 (OR output)	
D1 = X(Q1+Q0)	
D0 = X Q1' Q0'	

(b)

After the edge at t = 0, what is the earliest time at which D1 or D0 can change? Is the hold time met? How large could the hold time be before this circuit fails?

(c)

X comes from the rain sensor and can change at any time. Relative to a rising edge at t = 0, find the interval of time during which X must not change. Trace a change in X to each D input separately. Is X allowed to change 0.5 ns before the edge? What about 1.5 ns before it?

(d)

The controller runs with a 20 ns clock period. The sensor updates X exactly once per cycle, d ns after each rising edge ($0 \le d < 20$). For which values of d does the controller always store the correct values?

Q3 — One Register, Many Jobs

Question (discuss in class)

A 4-bit universal shift register holds A3 A2 A1 A0, where A3 is the leftmost bit (MSB). It has mode inputs S1 S0, a serial input SR that enters on a right shift, a serial input SL that enters on a left shift, and parallel inputs I3..I0. On every rising clock edge it does the following:

S1 S0	Operation on the rising edge
0 0	Hold: no change
0 1	Shift right: A3 ← SR, A2 ← A3, A1 ← A2, A0 ← A1
1 0	Shift left: A3 ← A2, A2 ← A1, A1 ← A0, A0 ← SL
1 1	Parallel load: A3..A0 ← I3..I0

(a)

Start with A = 0110, SR = 1, SL = 0 and I = 1001. Apply the modes 01, 01, 10, 00, 11, 10, one per clock edge, and fill in the table.

Edge	S1S0	A before	A after
1	01	0110	
2	01		
3	10		
4	00		
5	11		
6	10		

(b) Halving signed numbers

A shift left multiplies by 2. We would like a shift right to divide a 4-bit two's complement number by 2, giving $\lfloor N/2 \rfloor$ (the largest integer $\le N/2$). Fill in the table, writing the values in decimal.

N	N (4-bit)	$\lfloor N/2 \rfloor$	$\lfloor N/2 \rfloor$ (4-bit)	shift right, SR=0	value	shift right, SR=1	value
6							
5							
7							
-4							
-6							
-3							
-1							

Which SR value gives the correct answer for positive N? Which one works for negative N? Which existing signal should be wired to SR so that one right shift gives $\lfloor N/2 \rfloor$ for every N? With that wiring, does −3 become −1 or −2?

(c) A bouncing lamp

Four lamps are driven by A3..A0 (1 = lit). The register has already been loaded with 1000, and SR = SL = 0. A single lit lamp should move one position per clock edge, from left to right and back again: 1000, 0100, 0010, 0001, 0010, 0100, 1000, 0100, …

(i) Write S1S0 for each of the first 7 clock edges.

(ii) Instead of storing this sequence, add one D flip-flop Q that remembers the direction: Q = 0 means moving right (towards A0), and Q = 1 means moving left. The controller looks only at Q, A3 and A0. Fill in the table below. The case A3 = A0 = 1 cannot occur with one lamp lit, so treat it as a don't-care. Derive D, S1 and S0 with K-

maps. What do you notice about S1 and D? Is this controller a Mealy or a Moore machine? Its inputs are A3 and A0, and its outputs are S1 and S0.

Q	A3	A0	next Q (= D)	S1 S0
0	0	0		
0	0	1		
0	1	0		
1	0	0		
1	0	1		
1	1	0		

Q4 — Rain-Sensing Wipers: Two Ways to Wait

Question (optional — discuss in class if time permits, else take home)

A car has a rain sensor on the windscreen. Its output X is 1 when it detects water. The controller reads X once per clock cycle, at the rising clock edge. So that a single stray drop does not start the wipers, the wiper motor signal W should turn on only when rain has been seen in two consecutive clock cycles.

Two engineers built this with positive-edge-triggered D flip-flops that share one clock:

```
Design P (one flip-flop, state Q):        D  = X
                                          W  = X·Q

Design R (two flip-flops, state Q1Q0):     D1 = X·(Q1 + Q0)
                                          D0 = X·Q1'·Q0'
                                          W  = Q1
```

(a)

For each design, build the state table (present state, input X, next state, output W) and draw the state diagram. In one of the two diagrams you can write W inside each state circle; in the other you have to write it on the arrows, next to X. Which design is which, and what in its output equation tells you so?

Terminology: a machine whose output depends only on the present state is called a Moore machine. A machine whose output depends on the present state and the present input is called a Mealy machine. Label P and R accordingly.

(b)

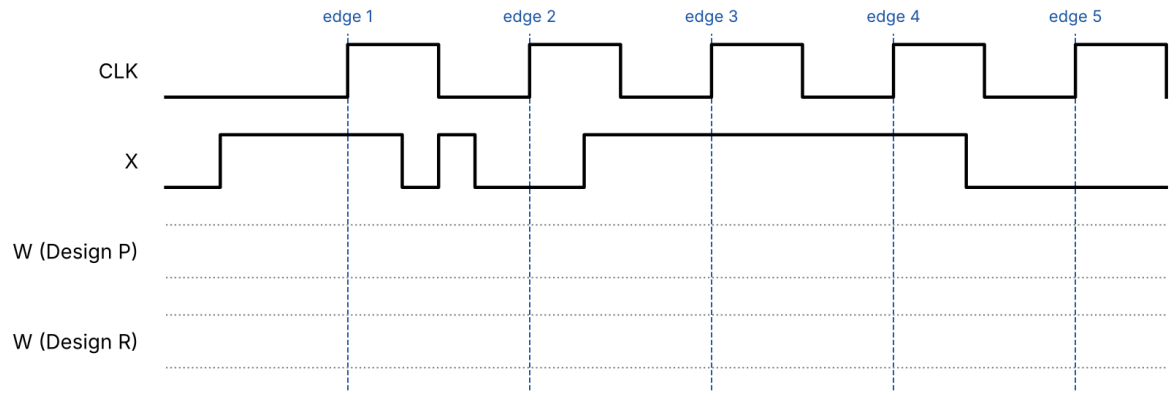
Both designs start in the all-zero state. In this part, X is held constant through each clock cycle and changes only just after a rising edge. Complete the table. Each column is one clock cycle, and the state shown is the state during that cycle.

Cycle	1	2	3	4	5	6	7	8	9	10	11	12
X	0	1	1	1	0	1	0	1	1	0	0	1
Q (P)												
W (P)												
Q1Q0 (R)												
W (R)												

Now compare the W (P) row with the W (R) row. State the relationship between them in one line.

(c)

Real sensors are not that tidy. In the waveform below, X changes at arbitrary times, and there is a brief splash between edges 1 and 2. Ignore all gate and flip-flop delays. Both designs are in the all-zero state before edge 1. Draw W for both designs.



Which design's motor sees the short splash? Does the splash change the stored state of either design? In which design can W change at a moment that is not a clock edge?

(d)

The customer now wants the wipers to start only after n consecutive rainy samples. Generalise the two designs (do not derive equations, just count). Fill in the table, then say for which values of n the Moore version needs one more flip-flop than the Mealy version.

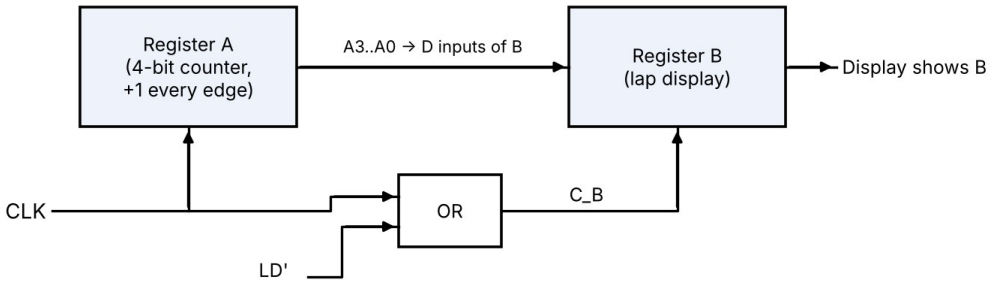
n (rainy samples in a row)	Mealy: states	Mealy: flip-flops	Moore: states	Moore: flip-flops
2				
3				
4				
5				
8				

Q5 — A Stopwatch Lap Button

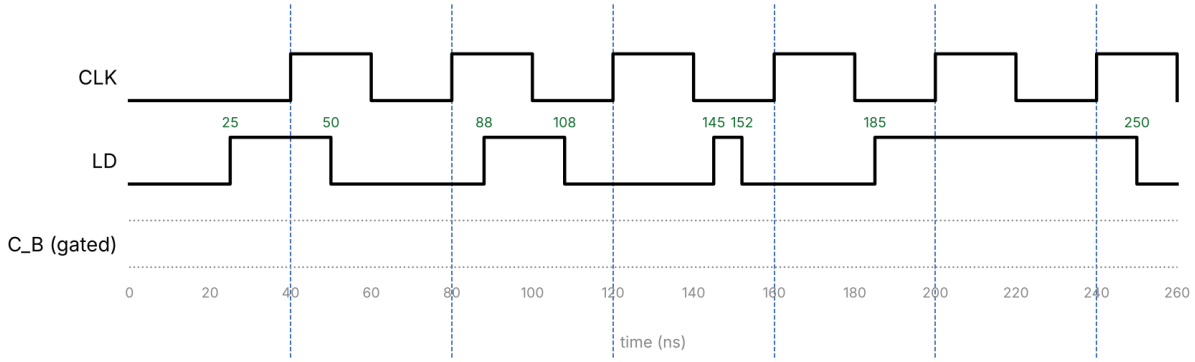
Question (optional — discuss in class if time permits, else take home)

In a stopwatch, register A is a 4-bit counter that adds 1 on every rising CLK edge. Register B drives the display. When the lap button is pressed, LD = 1. B should copy A only at a rising CLK edge where LD = 1, and hold its value otherwise.

A cheap way to do this is clock gating: B's clock input is C_B = CLK + LD'. When LD = 0, C_B is stuck at 1, so B never sees a rising edge.



CLK has a 40 ns period, with rising edges at 40, 80, 120, 160, 200 and 240 ns, and stays high for 20 ns after each edge. A = 3 before t = 40 and goes up by 1 at each edge. LD is shown below, with the times at which it changes.



(a) Ignore all delays

(i) Complete the table, which shows what a change in LD does to C_B for each CLK level.

CLK level	LD changes	C_B changes	Rising edge on C_B?
0	0 → 1		
0	1 → 0		
1	0 → 1		
1	1 → 0		

(ii) Draw C_B on the waveform and mark every rising edge.

(iii) Fill in when B loads and which value of A it captures, which is A's value just before that moment. Which button presses misbehave? From your table in (i), state the rule LD has to obey for the gated design to work correctly.

Design	Times at which B loads (ns)	Values loaded (decimal)
Intended behaviour		
Clock gating		

(b) Now include delays

Each flip-flop has a clock-to-Q delay between 3 and 5 ns, and different bits may differ. Setup time is 3 ns and hold time is 1 ns. If D changes inside this window, the stored bit is unpredictable: it can be either value. The OR gate delays B's clock edge by δ ns relative to CLK.

Look at the edge at t = 200, where A changes from 0111 to 1000 and LD = 1. For δ = 1, 4 and 9 ns, which value(s) could B end up holding? Could the display show 1111? 0000? What is the largest δ that guarantees B captures 0111?

(c) The fix

Instead, clock B directly from CLK and put a 2-to-1 multiplexer in front of each D input: D_Bi = LD·Ai + LD'·Bi, built as an AND (2 ns) followed by an OR (2 ns).

- (i) After an edge, what are the earliest and latest times at which a D input of B can change? Are hold and setup met with the 40 ns clock?
- (ii) LD also passes through the multiplexer. Relative to an edge, during what interval must LD not change? Does any LD change in the waveform above fall in that interval?
- (iii) When does B load now, and which values does it capture?